



CAMBRIDGE
INSTITUTE OF TECHNOLOGY

Project Presentation

On

“AI-powered Image and Video Editing”

By

K Prathyusha(1CD22CS059)

Satya Bonthala(1CD22CS137)

Vivek Sharma(1CD22CS187)

Under the Guidance of

Prof. Yashawini S

Department of
Computer Science & Engineering

www.cambridge.edu.in



Contents

- Introduction
- Objectives
- Literature Survey
- Proposed Method
- Hardware/Software Requirements
- Architecture
- Algorithms/Methodology
- Implementation Modules Identified
- Works completed
- Works to be completed.
- References

Introduction

- **Digital Era:** Multimedia content is everywhere.
- **Growing Demand:** Everyday users seek powerful editing tools.
- **Challenge:** Traditional software is complex.
- **Opportunity:** AI simplifies & enhances editing.

Introduction

- **Traditional Editing:** Requires technical skill & manual effort.
- **Steep Learning Curve:** Barrier for casual users.
- **AI's Impact:** Deep learning & computer vision automate/improve editing.

Introduction(Cont...)

- **AI Capabilities:** Analyzes content, applies intelligent transformations.
- **VidStyler Emerges:** Leverages AI for accessible, sophisticated editing.
- **Our Goal:** Bridge pro-grade editing with everyday user needs via intuitive interfaces.

Objectives

Primary Goal: Develop "**VidStyler**" - an integrated, AI-powered, accessible editing suite.

Specific Aims:

- **Neural Style Transfer:** Robust style application, preserving content.
- **Video Stabilization:** Reduce shake with customizable smoothing.
- **Intelligent Object Removal:** Seamlessly remove elements, plausibly fill gaps.

Objectives (Cont...)

Further Aims:

- **Intuitive UI:** User-friendly interface, no deep technical knowledge needed.
- **Practical Performance:** Balance quality & speed for standard hardware.
- **Media Support:** Common formats, varying resolutions.
- **Customization:** User control over key parameters.
- **Documentation:** Share methodologies and learnings.

Literature Review - Image Style Transfer

Traditional: Stroke-based rendering, filtering, texture synthesis.

Neural Style Transfer (NST):

- **CNN-based (Slow but Accurate):** Early methods.
- **Optimized for Speed:** Fast Neural Methods.
- **GAN-based:** Generative Adversarial Networks.
- **Arbitrary Style:** Flexible style-content pairing (e.g., via dual attention).
- **Enhancements:** Multi-style, Real-time, Neural Neighbor for realism.

Literature Review - Video Stabilization

- **General Methods:**

- **Early:** Filtered camera paths for jitter reduction.
- **Advanced:** Optimized camera paths for cinematic smoothness.
- **Efficiency:** Integration with video encoding.

- **Special Cases:**

- **Airborne Vision:** Addresses drone jitter.
- **Selfie Videos:** Dual-branch networks for separate foreground/background stabilization.

Literature Review - Object Removal

Core Idea: Remove objects, maintain visual coherence.

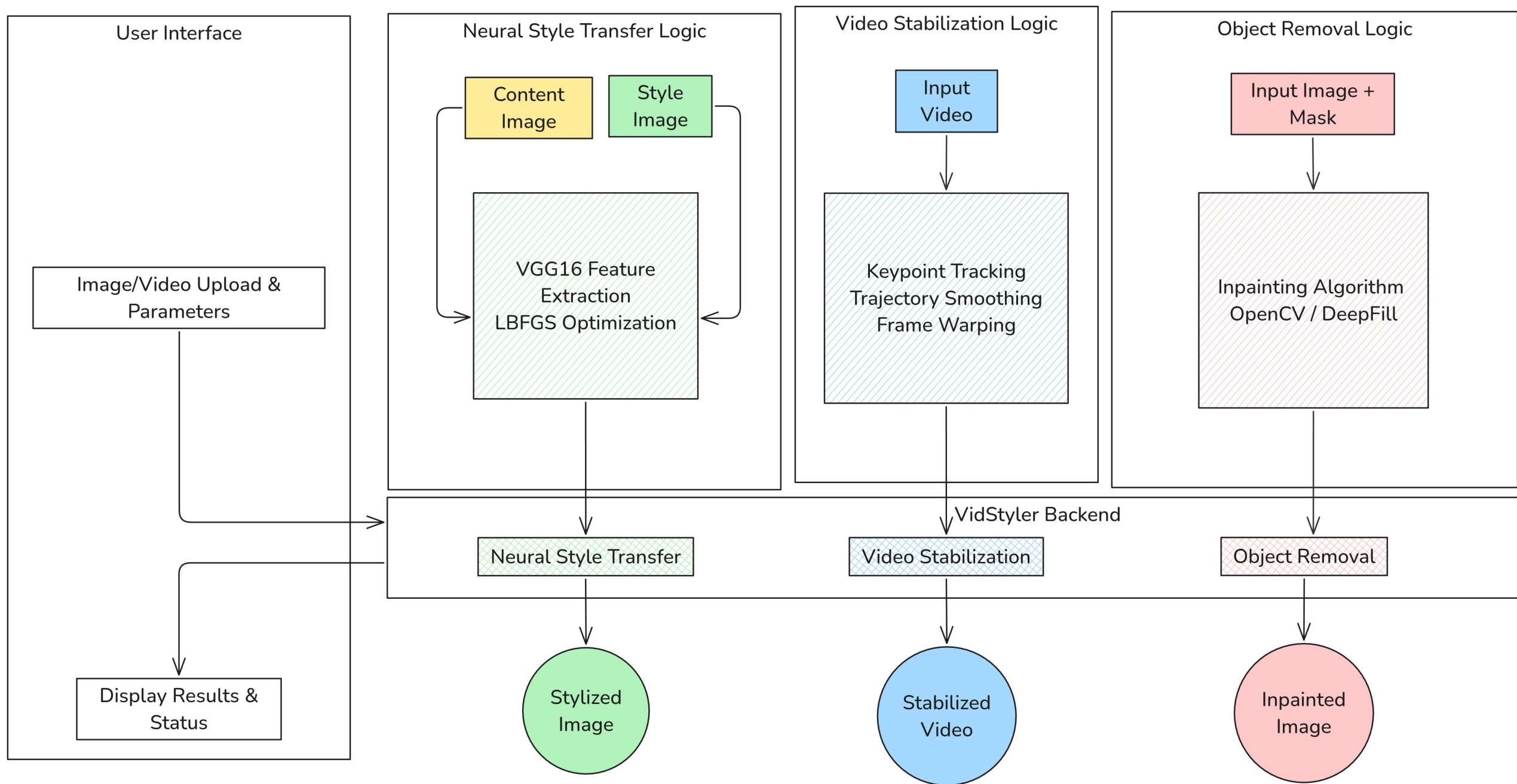
- **Autonomous Driving Example:** DR-REMOVER for dynamic objects in 3D maps.

Evaluation:

- **Traditional:** Compare with original images.
- **Newer:** Use images without the target object class for evaluation.

Proposed method

- **VidStyler Suite:** AI-driven editing for images & videos.
- **Core Features:**
 - **Neural Style Transfer:** Artistic image transformation.
 - **Video Stabilization:** Smooth shaky footage.
 - **Object Removal:** Clean unwanted elements from images.
- **Unified UI:** Gradio-based web interface.
- **Technology Stack:** Python, PyTorch, TensorFlow, OpenCV.



Proposed Method (Cont...) - Feature Details

- **Neural Style Transfer:**

- VGG16 for content/style features.
- Gram matrices for style.
- LBFGS optimization minimizes content & style loss (MSE).
- User controls: weights, iterations.

- **Video Stabilization:**

- Keypoint tracking (GFTT, SIFT, etc.).
- Smoothed trajectory via moving average.
- Frame warping, border handling, optional artistic effects.

Proposed Method (Cont...) - Object Removal

Object Removal:

- Mask input via Gradio ImageEditor.
- **Inpainting methods:**
 - **OpenCV:** Traditional (Navier-Stokes, Telea).
 - **DeepFill:** ML-based (TensorFlow model) for complex scenes.
- Real-time status feedback.

Hardware/Software requirements

- **Hardware:**

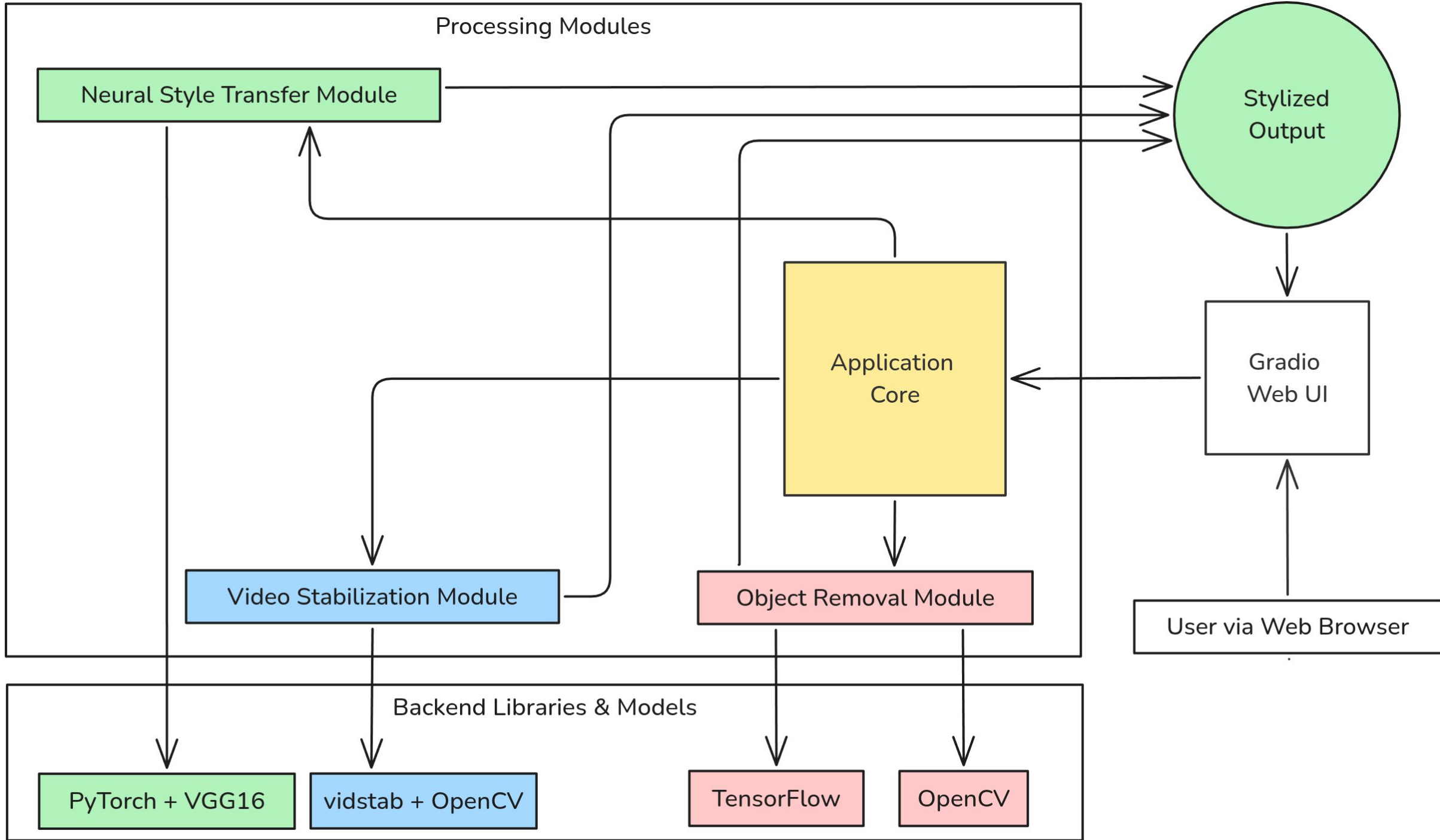
- Processor: Multi-core (e.g., Intel i5 11th Gen+)
- RAM: 8 GB+
- Storage: SSD (256 GB+ free)
- GPU (Recommended): NVIDIA CUDA-enabled (for faster DL tasks)

- **Software:**

- OS: Windows 10/11, macOS, Linux
- Core: Python 3.10+
- Libraries: PyTorch, TensorFlow, OpenCV, Gradio, NumPy, Matplotlib
- IDE: Visual Studio Code

Architecture

- **Model:** Modular Layered Design (MVC-like).
- **UI (View):** Gradio web interface.
- **Logic (Controller):** Bridges UI input to Model functions.
- **Data/Models (Model):** Data structures, DL models.
- **Simplifies:** Debugging, testing, feature updates.
- **Core System Modules:**
 - Neural Style Transfer
 - Video Stabilization
 - Object Removal



Architecture (Cont...) - Module Breakdown

•Neural Style Transfer:

- VGG16 feature extraction.
- Gram matrices for style.
- Adjustable weights.

•Video Stabilization:

- Multiple keypoint detectors (ORB, SIFT, GFTT).
- Frame-to-frame transform smoothing.
- Border handling, motion trails.

•Object Removal:

- User-drawn masks.
- DeepFill (ML) or OpenCV (traditional) inpainting.
- Real-time feedback.



Architecture (Cont...) - Project Structure & Tech Stack

•Project Structure:

- app/: Gradio UI.
- src/: Core logic (style_transfer, video_stabilization, object_removal).
- run.py: Entry point.

•Key Architectural Features:

- Clear separation of concerns.
- Modular, testable code.
- Extensible.

•Technology Stack:

- Language: Python
- Deep Learning: PyTorch, TensorFlow
- UI: Gradio
- CV: OpenCV
- Optimizer (NST): LBFGS
- GPU Support: CUDA (recommended)

Algorithms

- **Neural Style Transfer:**
 - **VGG16:** Feature extraction.
 - **Gram Matrix:** Style representation.
 - **MSE Loss:** Content & Style difference.
 - **L-BFGS:** Optimization algorithm.
- **Video Stabilization:**
 - **Keypoint Detection:** GFTT, SIFT, ORB, etc.
 - **Motion Estimation & Smoothing:** Moving average on transforms.
- **Object Removal:**
 - **Masking:** User input + image processing.
 - **Inpainting:** OpenCV (NS, Telea), DeepFill (CNN-based).

Implementation Modules

1. Application Core (app/app.py):

- Gradio UI integration.
- Input handling, calls backend logic.

2. Neural Style Transfer (src/style_transfer/):

- neural_style.py: Main NST logic, L-BFGS loop.
- models.py: VGG16 definition.

3. Video Stabilization (src/video_stabilization/):

- stabilize.py: UI to VidStabWrapper interface.
- vidstab.py: Wrapper for vidstab library.

Implementation- Modules(Cont...)

4. Object Removal (src/object_removal/):

- inpainting.py: Selects & applies OpenCV/DeepFill.
- deepfill_inpainter.py: Manages DeepFill TensorFlow model.
- model/model.py: DeepFill CNN architecture.

5. Utilities & Setup:

- run.py: App launcher.
- requirements.txt: Dependencies.
- README.md: Documentation.

Works Completed

- **Core Features Implemented:**

- Neural Style Transfer (VGG16, LBFGS).
- Video Stabilization (vidstab, keypoint methods, plotting).
- Object Removal (Masking, OpenCV & DeepFill inpainting).

- **Gradio UI Developed:**

- Tabbed interface for each function.
- Interactive controls & feedback.

- **Project Foundation:**

- Modular structure & initial documentation.

Works to be completed

- **Refinement & Optimization:**
 - Comprehensive Testing (unit, integration).
 - Performance Profiling & Optimization (NST, DeepFill).
- **Feature Enhancements:**
 - Video style transfer; Video object removal.
 - Automatic mask generation (object detection).
- **Usability & Management:**
 - Streamline DeepFill model handling.
 - UI/UX improvements.
 - Expand documentation & create deployment guides.

References

Key Libraries: Gradio, PyTorch, TensorFlow, OpenCV, VidStab, NumPy.

Key Concepts & Inspirations (Underlying Research):

- Neural Style Transfer: Gatys, L. A., Ecker, A. S., & Bethge, M. (2016).
- Image style transfer using convolutional neural networks. CVPR.
- Deep Image Inpainting (e.g., DeepFill): Yu, J., Lin, Z., Yang, J., Shen, X., Lu, X., & Huang, T. S. (2018).
- Generative image inpainting with contextual attention.
- CVPR. (And subsequent DeepFill v2)
- Feature-Based Video Stabilization: Techniques involving keypoint detection (SIFT, SURF, ORB, etc.), feature tracking, and motion smoothing. (General field of research).

Thank you